

Reviewer Pack — Minimal p-Adic Valuation Rank Bound for SAT Clause Complexit...

Entry a66d82f17ea4 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-08 08:48:15 UTC

Minimal p-Adic Valuation Rank Bound for SAT Clause Complexity

Entry ID: a66d82f17ea4 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-08 08:48:15 UTC

1. Conjecture

Field A (mathematical branch): p-Adic Number Theory **Field B** (complexity object): Boolean Satisfiability (Clause Complexity)

Statement:

For every Boolean formula F with m clauses, its clause complexity $c(F)$ is bounded by the minimal p-adic valuation rank $v(p)(F)$, where $v(p)(F)$ is the smallest integer such that $p^{v(p)(F)}$ divides at least one coefficient in the characteristic polynomial of F over $\mathbb{Z}_p$.

Rationale (proposer's reasoning):

p-Adic valuation ranks can provide a finer measure of algebraic complexity, which may reveal subtle relationships between arithmetic properties and computational hardness. This conjecture proposes a potential link between an arithmetic invariant and clause complexity, suggesting that certain arithmetic complexities could imply limitations on the structure of Boolean satisfiability problems.

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ede5776a8645b4e3

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the linear regression analysis across multiple seeds reveals a statistically significant positive correlation (p-value < 0.05) with a coefficient of determination $R^2 \geq 0.7$ between clause complexity and minimal p-adic valuation rank, for all $m \leq 40$ clauses.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 3 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "p-adic valuation rank" AND "Boolean satisfiability clause complexity" - "characteristic polynomial" AND $\mathbb{Z}_p$ AND "SAT clause complexity" - "minimal p-adic valuation rank" IN BOOLEAN LOGIC AND "SAT problem"

Top relevant hits considered: - [http://arxiv.org/abs/1102.1242v2] A refinement of Stone duality to skew Boolean algebras - [http://arxiv.org/abs/0903.3848v1] Join-irreducible Boolean functions - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=0, elapsed=3.1s

5.1 Generated Python source

```
import random
from fractions import Fraction
import math
import sys

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(A):
        n = len(A)
        for i in range(n):
            max_row = i
            for j in range(i + 1, n):
                if abs(A[j][i]) > abs(A[max_row][i]):
                    max_row = j
            A[i], A[max_row] = A[max_row], A[i]
            pivot = A[i][i]
            if pivot == 0:
                continue
            for j in range(i, n):
                A[i][j] /= pivot
            for k in range(n):
                if k != i and A[k][i] != 0:
                    factor = A[k][i]
                    for j in range(i, n):
                        A[k][j] -= factor * A[i][j]
        return A

    def characteristic_polynomial(F, p):
```

```

n = len(F)
A = [[Fraction(0) for _ in range(n)] for _ in range(n)]
for i in range(n):
    for j in range(n):
        if i == j:
            A[i][j] += Fraction(-1)
        else:
            A[i][j] += Fraction(F[i][j], p)
A = gaussian_elimination(A)
det = 1
for i in range(n):
    det *= A[i][i]
return det

def clause_complexity(F):
    m = len(F)
    return m

n_values = [5, 10, 15, 20, 30, 40]
metrics = []

for n in n_values:
    F = [[random.randint(0, 1) for _ in range(n)] for _ in range(n)]
    p = random.choice([2, 3, 5, 7, 11])
    v_p = characteristic_polynomial(F, p)
    c_F = clause_complexity(F)
    metrics.append({
        "n": n,
        "v_p": v_p,
        "c_F": c_F
    })

if not metrics:
    return {
        "metric_name": "min_v_p",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "no_metrics_collected"
    }

min_v_p = min(metric["v_p"] for metric in metrics)
mean_c_F = sum(metric["c_F"] for metric in metrics) / len(metrics)
std_dev_c_F = math.sqrt(sum((metric["c_F"] - mean_c_F) ** 2 for metric in metrics) / len(metrics))

return {
    "metric_name": "min_v_p",
    "metric_value": min_v_p,
    "instances_tested": len(metrics),
    "n_max": max(metric["n"] for metric in metrics),
    "conjecture_holds": False,
    "counterexample": "not_computed"
}

if __name__ == "__main__":

```


9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code does not compute the characteristic polynomial correctly for Boolean formulas, which is a critical part of the conjecture's definition. The critic has challenged the results due to an incorrect implementation. | next: Revise the test code to correctly calculate the characteristic polynomial and retest the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15437
2	preregistration	ollama_remote	glm4:latest	0	0	10624
3	novelty	ollama_remote	glm4:latest	0	0	8777
4	novelty	ollama_remote	glm4:latest	0	0	18420
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	45780
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10908
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12148
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12416
9	critic	ollama_remote	glm4:latest	0	0	15877
10	judge	ollama_remote	glm4:latest	0	0	14848

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 165234 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/a66d82f17ea4.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/a66d82f17ea4.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/a66d82f17ea4.tar.gz (if generated)